

Does this apply to you? — one minute

If your team runs an agentic coding assistant with broad permissions, the answer is probably yes. The questions below determine *how strict* your configuration should be, not whether to bother.

|                          |                                                                                                                                                                                 |    |
|--------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| <input type="checkbox"/> | <b>Does this repository handle personal, financial, or otherwise regulated data?</b><br>Customer records, payment data, health information, anything under a regulatory regime. | E1 |
| <input type="checkbox"/> | <b>Does it deploy to production?</b><br>Directly, or through a pipeline the agent can trigger.                                                                                  | E2 |
| <input type="checkbox"/> | <b>Is it public-facing or open source?</b><br>Errors are externally visible, and externally exploitable.                                                                        | E3 |
| <input type="checkbox"/> | <b>Does it ingest untrusted third-party content?</b><br>Dependency docs, user-submitted issues, scraped data. This is the prompt injection pathway.                             | E4 |
| <input type="checkbox"/> | <b>Is it subject to a specific audit or compliance obligation?</b><br>Evidentiary and attribution requirements are stricter.                                                    | E5 |

Every box you ticked raises every action one tier, capped at approval. Set the corresponding flag to true in .amp/tier-policy.json. Two or more ticks means routine actions will require approval — expect friction, and plan to tune. Zero ticks is still worth running: the baseline blocks things no agent should do anywhere.

Rolling it out

Do not switch straight to enforcement. You will not know what it breaks, and a control that surprises people gets disabled rather than tuned.

- 1

**Install and set observation mode**  
Copy the plugin and config into .amp/, set "mode": "observe", reload. Nothing is blocked. Every decision is recorded.
- 2

**Work normally for two weeks**  
Don't change how you use the agent. Capture what your workflow actually looks like.
- 3

**Read the log**  
Run show-policy-summary. Look at the wouldHave breakdown: how many actions would have been blocked, how many would have prompted, and which rules fire most.
- 4

**Tune before enforcing**  
Any rule producing a high volume of approval prompts is a candidate for an override. A control that fires constantly trains people to click through it, which is worse than not having it. Record each override in the config so the deviation is explicit.
- 5

**Switch to enforcement**  
Set "mode": "enforce". Re-read the log after a week. If approval grant rate is approaching 100% and response times are collapsing, people have stopped reading the prompts and you should tighten scope rather than celebrate.

What you get, and what you don't

| Enforced                                                       | Not covered                                                    |
|----------------------------------------------------------------|----------------------------------------------------------------|
| Force pushes, protected-branch writes, repository deletion     | Content typed into a prompt — the policy sees tool calls only  |
| Credential and secret file access, environment enumeration     | Actions expressed indirectly, through encoding or substitution |
| Modification of the agent's own configuration or the audit log | Whether an approved action subsequently succeeded              |
| Production deployment, infrastructure and IAM changes          | Whether the configuration is appropriate for the repository    |

**One thing worth knowing before you start**

A model with its own safety behaviour sometimes declines an action before the policy sees it, and an agent that has been blocked once learns the policy from the rejection message. Counts of prevented actions drift downward over a long session without the control having weakened. Judge the policy by its decision log, not by what you observe the agent doing.

|   |                                                                                                                                                                          |                                                                                                                                                 |
|---|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------|
| 1 | <b>Would this let the agent change what it is allowed to do?</b><br>Editing its instruction file, installing a connector, modifying the policy config, altering logging. | <b>T4 Prohibited.</b><br>An agent that can grant itself authority has no constraints. Approving it once removes the ability to refuse it later. |
| 2 | <b>Is it irreversible, or reversible only with significant effort?</b><br>Force pushes, repository deletion, dropped tables, applied infrastructure changes.             | <b>T4 Prohibited.</b><br>If a mistaken approval cannot be undone, the prompt is a formality before an unrecoverable outcome.                    |
| 3 | <b>Does it read or write credentials, secrets, or real customer data?</b><br>Credential stores, key files, environment variables, production databases.                  | <b>T4 Prohibited.</b><br>No legitimate workflow requires direct credential access. If one appears to, change the workflow.                      |
| 4 | <b>Does it reach production, or leave organisational control?</b><br>Deployments, outbound network calls, third-party APIs, data sent externally.                        | <b>T3 Approval required</b> — or T4 if the destination is unapproved.<br>A human should know before something crosses a boundary.               |
| 5 | <b>Does it change what other people will run?</b><br>Adding a dependency, changing a lockfile, modifying git hooks, rewriting shared history.                            | <b>T3 Approval required.</b><br>Reversible, but propagates quickly, and the person affected is rarely the one who approved it.                  |
| 6 | <b>Does the effect leave this machine, or become visible to others?</b><br>Pushing a branch, opening a pull request, deleting a tracked file.                            | <b>T2 Notify.</b><br>Recoverable, but someone should be able to see it happened without going looking.                                          |
| 7 | <b>Is it contained, reversible, and frequent?</b><br>Reading files, editing in the workspace, running tests, local commits.                                              | <b>T1 Autonomous.</b><br>The overwhelming majority of agent activity. Interrupting it is how a control trains people to ignore it.              |
| 8 | <b>None of the above, or you are unsure.</b><br>An unfamiliar command, or a tool you have not seen the agent use.                                                        | <b>T3 Approval required.</b><br>Unknown means approval, never autonomous. A gap should produce friction you notice, not silence you don't.      |

Then apply context

Raise the result one tier for each condition true of the repository: regulated data (E1), production deployment (E2), public-facing (E3), ingests untrusted content (E4), audit obligation (E5).

**Escalation stops at approval.** Tier 4 comes from the ladder above, never from accumulating context — otherwise a sufficiently sensitive repository would forbid an agent from reading a file.

Two questions worth asking before you finalise a tier

Would you switch this off? If the honest answer is yes, the tier is too high. A control people route around provides no protection and false assurance, which is worse than having none.

Would you notice if it were missing? If the honest answer is no, the tier may be too low — or the action may not need a rule at all.

Adding it to the rule set

Give the action an ID from its taxonomy category (A filesystem, B execution, C dependencies, D version control, E review, F network, G secrets, H data, I CI/CD, J infrastructure, K self-configuration, L audit), write patterns for both shell dialects, and place it **above** any broader rule that would match first. Rules run narrowest to broadest; the catch-all stays last.

Add a test case with the expected rule ID and tier before you commit. Rule ordering is the property most likely to break silently, and a misplaced rule produces a policy far stricter or far looser than its author intended.